

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»
Дисциплина «Основы программной инженерии»

Лабораторная работа №4
Вариант №489356

Выполнил:
Пивоваров Р.Н.

Проверил:
Кобик Н.А.

Дата сдачи: 13 Июня
Санкт-Петербург, 2026

Содержание

1. Задание	3
2. исходный код Mbeans и вспомогательных классов	4
3. Jconsole	8
3.1. снимаем показания Mbean	8
3.2. Определить значение переменной classpath для данной JVM.	10
4. VisualVM	10
4.1. Снять график изменения показаний MBean-классов	10
4.2. определить имя класса, объекты которого занимают наибольший объём памяти JVM	11
4.3. определить пользовательский класс, в экземплярах которого находятся эти объекты	12
5. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе	13
6. Заключение	15

1. Задание

1. Для своей программы из лабораторной работы 3 по дисциплине «Веб-программирование» реализовать:
 - MBean, считающий общее число установленных пользователем точек, а также число точек, попадающих в область. В случае, если пользователь совершил 2 «промаха» подряд, разработанный MBean должен отправлять оповещение об этом событии.
 - MBean, определяющий процентное отношение «попаданий» к общему числу кликов пользователя по координатной плоскости.
2. С помощью утилиты JConsole провести мониторинг программы:
 - Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
 - Определить значение переменной classpath для данной JVM.
3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:
 - Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
 - Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.
4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:
 - Описание выявленной проблемы.
 - Описание путей устранения выявленной проблемы.
 - Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.
 - Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Отчёт по работе должен содержать:

- Текст задания.
- Исходный код разработанных MBean-классов и сопутствующих классов.
- Скриншоты программы JConsole со снятыми показаниями, выводы по результатам мониторинга.
- Скриншоты программы VisualVM со снятыми показаниями, выводы по результатам профилирования.
- Скриншоты программы VisualVM с комментариями по ходу поиска утечки памяти.
- Выводы по работе.

2. исходный код Mbeans и вспомогательных классов PointsStatCounterMBean.java

```
package app.backend.MyMbeans;

public interface PointsStatCounterMBean {
    int getTotalPoints();
    int getHitPoints();
}
```

PointsStatCounter.java

```
package app.backend.MyMbeans;

import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;
import java.util.concurrent.atomic.AtomicInteger;

public class PointsStatCounter extends
NotificationBroadcasterSupport implements PointsStatCounterMBean {
    private final AtomicInteger totalPoints = new AtomicInteger(0);
    private final AtomicInteger hitPoints = new AtomicInteger(0);
    private final AtomicInteger consecutiveMisses = new
AtomicInteger(0);
    private long sequence = 0;

    public void registerAttempt(boolean isHit) {
        totalPoints.incrementAndGet();
    }
}
```

```

        if (isHit) {
            hitPoints.incrementAndGet();
            consecutiveMisses.set(0);
        } else {
            int misses = consecutiveMisses.incrementAndGet();
            if (misses >= 2) {
                MissAlert();
                consecutiveMisses.set(0);
            }
        }
    }

private void MissAlert() {
    Notification notification = new Notification(
        "jmx.alert.consecutiveMisses",
        this,
        sequence++,
        System.currentTimeMillis(),
        "Пользователь совершил 2 промаха подряд"
    );
    sendNotification(notification);
}

@Override
public int getTotalPoints() {
    return totalPoints.get();
}

@Override
public int getHitPoints() {
    return hitPoints.get();
}

@Override
public MBeanNotificationInfo[] getNotificationInfo() {
    String[] types = {"jmx.alert.consecutiveMisses"};
    String name = Notification.class.getName();
    String description = "Оповещение о двух промахах подряд";
    return new MBeanNotificationInfo[]{
        new MBeanNotificationInfo(types, name, description)
    };
}
}

```

PointsStatAnalysisMBean.java

```
package app.backend.MyMbeans;

public interface PointsStatAnalysisMBean {
    double getHitPercentage();
}
```

PointsStatAnalysis.java

```
package app.backend.MyMbeans;

public class PointsStatAnalysis implements PointsStatAnalysisMBean{
    private final PointsStatCounter pointsCounter;

    public PointsStatAnalysis(PointsStatCounter pointsCounter) {
        this.pointsCounter = pointsCounter;
    }

    @Override
    public double getHitPercentage() {
        int total = pointsCounter.getTotalPoints();
        if (total == 0) {
            return 0.0;
        }
        return (pointsCounter.getHitPoints() * 100.0) / total;
    }
}
```

MBeanInitializer.java

```
package app.backend.MyMbeans;

import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;
import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

@WebListener
public class MBeanInitializer implements ServletContextListener {
```

```

    public static final String COUNTER_ATTR =
"PointsStatCounterMBean";

    private ObjectName counterName;
    private ObjectName ratioName;

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        try {
            MBeanServer mbs =
ManagementFactory.getPlatformMBeanServer();

            counterName = new
ObjectName("lab3.jmx:type=PointsStatCounter");
            PointsStatCounter counter = new PointsStatCounter();
            mbs.registerMBean(counter, counterName);

            ratioName = new
ObjectName("lab3.jmx:type=PointsStatAnalysis");
            PointsStatAnalysis ratio = new
PointsStatAnalysis(counter);
            mbs.registerMBean(ratio, ratioName);

            sce.getServletContext().setAttribute(COUNTER_ATTR,
counter);

        } catch (Exception e) {
            throw new RuntimeException("Не удалось зарегистрировать
MBean", e);
        }
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        try {
            MBeanServer mbs =
ManagementFactory.getPlatformMBeanServer();
            mbs.unregisterMBean(counterName);
            mbs.unregisterMBean(ratioName);
        } catch (Exception ignored) {
        }
    }
}

```

3. Jconsole

3.1. снимаем показания Mbean

Запускаем jconsole и подключаемся к процессу с jboss в названии (т.к. я использую wildfly) заходим на вкладку Mbeans и находим наши бины

The screenshot shows the JConsole interface with the 'MBeans' tab selected. The left sidebar displays a tree of MBeans, with 'lab3.jmx' expanded and 'PointsStatAnalysis' selected. The main pane shows the 'MBeanInfo' for this MBean.

Name	Value
Info:	
ObjectName	lab3.jmx:type=PointsStatAnalysis
ClassName	app.backend.MyMbeans.PointsStatAnalysis
Description	Information on the management interface of the MBean
Constructor-0:	
Name	app.backend.MyMbeans.PointsStatAnalysis
Description	Public constructor of the MBean
Parameter-0-0:	
Name	p1
Description	
Type	app.backend.MyMbeans.PointsStatCounter

Name	Value
Descriptor	
Info:	
immutableInfo	true
interfaceClassName	app.backend.MyMbeans.PointsStatAnalysisMBean
mxbean	false

отправляем пару точек и проверяем снова

The screenshot shows the JConsole interface with the 'MBeans' tab selected. The left sidebar is the same as the previous image. The main pane now shows the 'Attribute values' for the selected MBean.

Name	Value
HitPercentage	66.66666666666667

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

Attribute values

Name	Value
HitPoints	2
TotalPoints	3

подписываемся на уведомления и делаем 2-ой промах, получаем уведомление

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

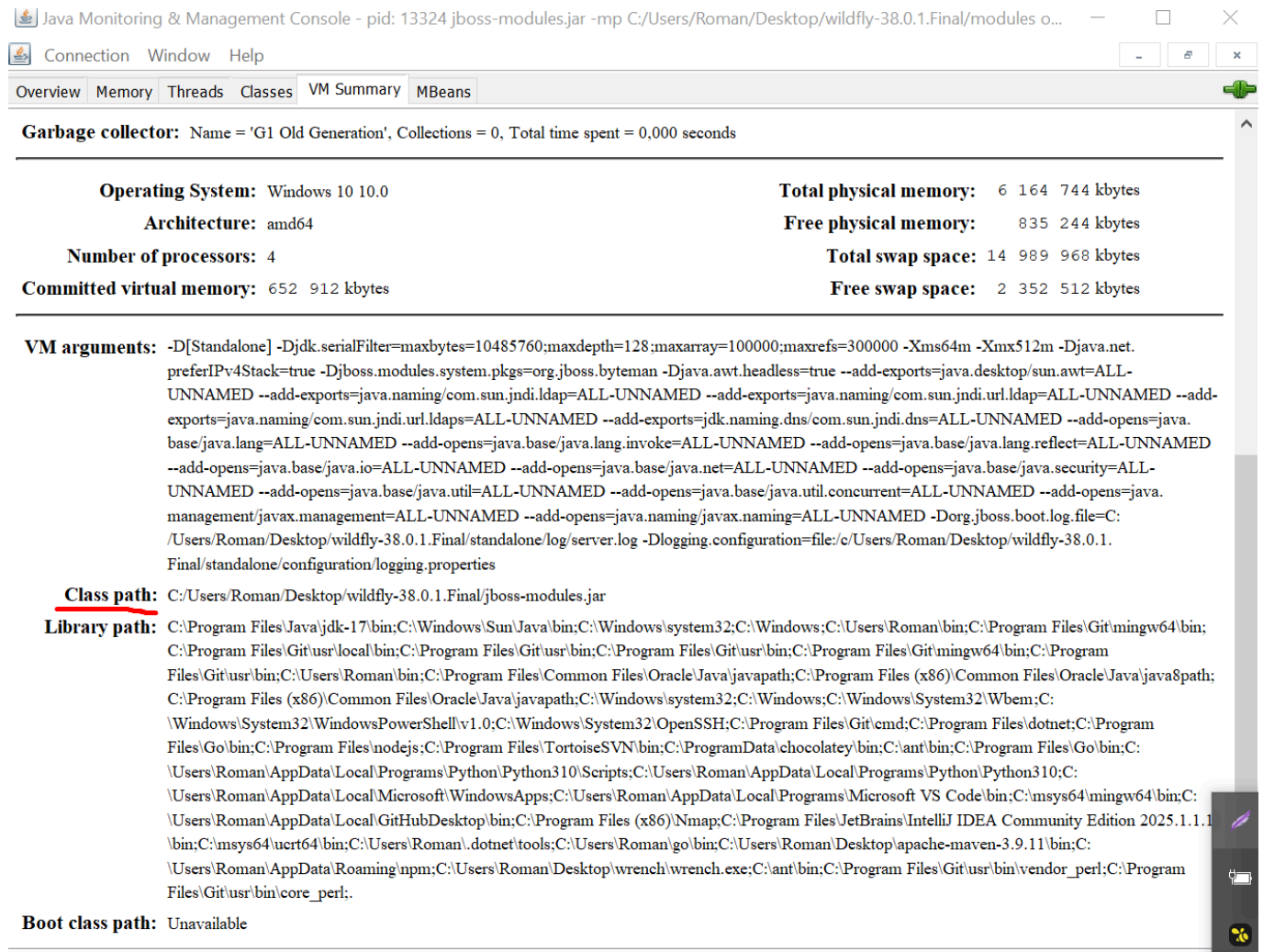
Notification buffer

TimeStamp	Type	UserData	SeqNum	Message	Event	Source
09:11:20:297	jmx.alert.consecu...		0	Пользователь совершил 2 промаха...	javax.man...	lab3.jmx:t...

Subscribe Unsubscribe Clear

3.2. Определить значение переменной classpath для данной JVM.

открываем вкладку VMsummary и находим там classpath



Java Monitoring & Management Console - pid: 13324 jboss-modules.jar -mp C:/Users/Roman/Desktop/wildfly-38.0.1.Final/modules o...

Connection Window Help

Overview Memory Threads Classes VM Summary MBeans

Garbage collector: Name = 'G1 Old Generation', Collections = 0, Total time spent = 0,000 seconds

Operating System: Windows 10 10.0	Total physical memory: 6 164 744 kbytes
Architecture: amd64	Free physical memory: 835 244 kbytes
Number of processors: 4	Total swap space: 14 989 968 kbytes
Committed virtual memory: 652 912 kbytes	Free swap space: 2 352 512 kbytes

VM arguments: -D[Standalone] -Djdk.serialFilter=maxbytes=10485760;maxdepth=128;maxarray=100000;maxrefs=300000 -Xms64m -Xmx512m -Djava.net.preferIPv4Stack=true -Djboss.modules.system.pkgs=org.jboss.byteman -Djava.awt.headless=true --add-exports=java.desktop/sun.awt=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi ldap=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url ldap=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url ldaps=ALL-UNNAMED --add-exports=java.naming/com.sun.jndi.url ldap=ALL-UNNAMED --add-exports=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-opens=java.base/java.net=ALL-UNNAMED --add-opens=java.base/java.security=ALL-UNNAMED --add-opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED --add-opens=java.management/javax.management=ALL-UNNAMED --add-opens=java.naming/javax.naming=ALL-UNNAMED -Dorg.jboss.boot.log.file=C:/Users/Roman/Desktop/wildfly-38.0.1.Final/standalone/log/server.log -Dlogging.configuration=file:/c:/Users/Roman/Desktop/wildfly-38.0.1.Final/standalone/configuration/logging.properties

Class path: C:/Users/Roman/Desktop/wildfly-38.0.1.Final/jboss-modules.jar

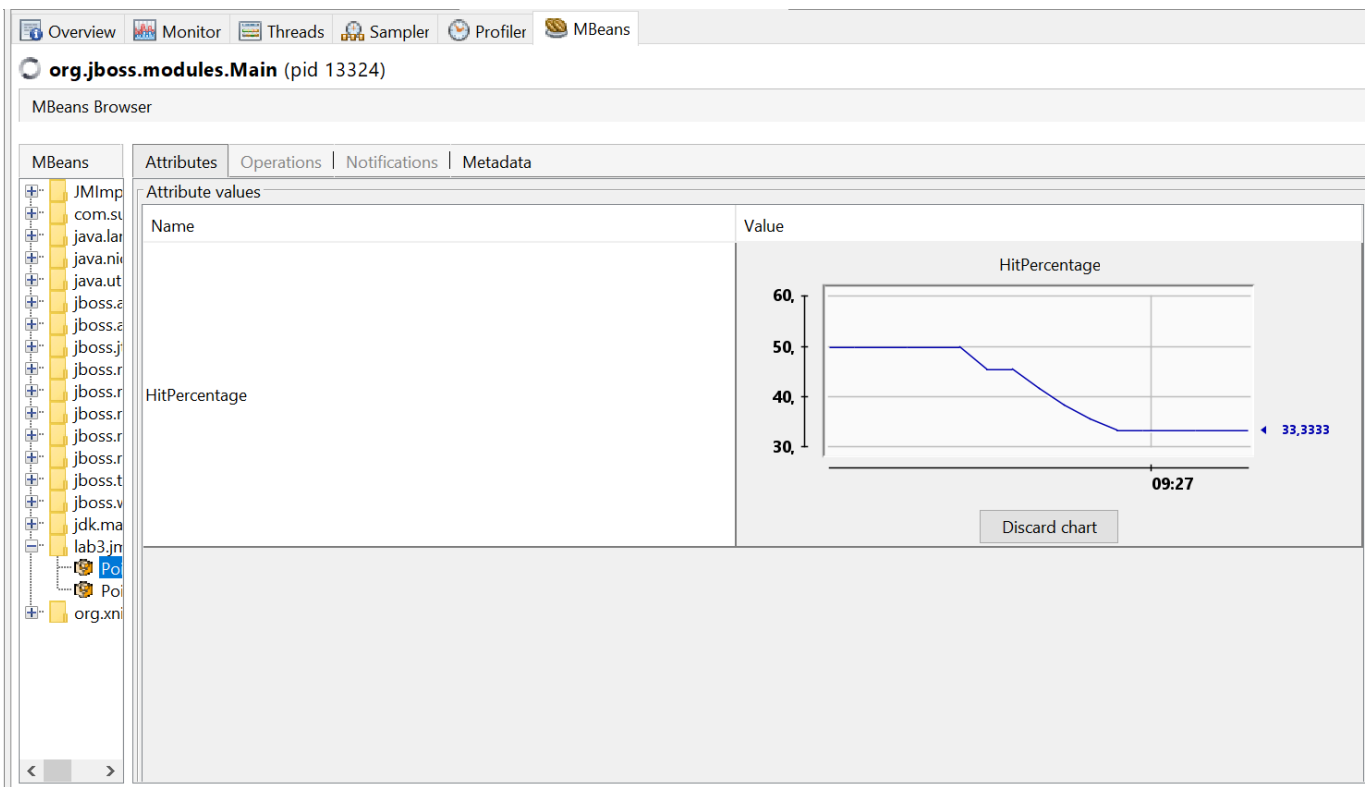
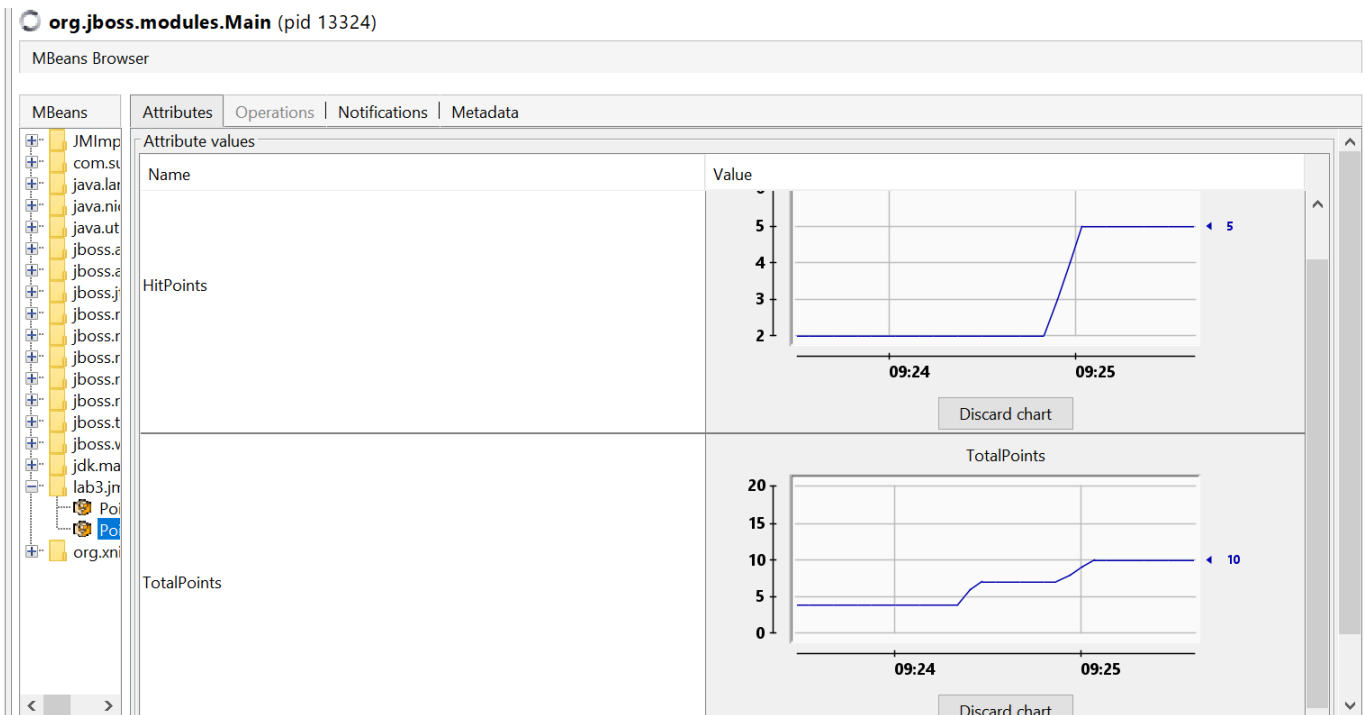
Library path: C:\Program Files\Java\jdk-17\bin;C:\Windows\Sun\Java\bin;C:\Windows\system32;C:\Windows;C:\Users\Roman\bin;C:\Program Files\Git\mingw64\bin;C:\Program Files\Git\usr\local\bin;C:\Program Files\Git\usr\bin;C:\Program Files\Git\mingw64\bin;C:\Program Files\Git\usr\bin;C:\Users\Roman\bin;C:\Program Files\Common Files\Oracle\Java\javapath;C:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0;C:\Windows\System32\OpenSSH;C:\Program Files\Git\cmd;C:\Program Files\dotnet;C:\Program Files\Go\bin;C:\Program Files\nodejs;C:\Program Files\TortoiseSVN\bin;C:\ProgramData\chocolatey\bin;C:\ant\bin;C:\Program Files\Go\bin;C:\Users\Roman\AppData\Local\Programs\Python\Python310\Scripts;C:\Users\Roman\AppData\Local\Programs\Python\Python310;C:\Users\Roman\AppData\Local\Microsoft\WindowsApps;C:\Users\Roman\AppData\Local\Programs\Microsoft VS Code\bin;C:\msys64\mingw64\bin;C:\Users\Roman\AppData\Local\GitHubDesktop\bin;C:\Program Files (x86)\Nmap;C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.1\bin;C:\msys64\ucrt64\bin;C:\Users\Roman\dotnet\tools;C:\Users\Roman\go\bin;C:\Users\Roman\Desktop\apache-maven-3.9.11\bin;C:\Users\Roman\AppData\Roaming\npm;C:\Users\Roman\Desktop\wrench\wrench.exe;C:\ant\bin;C:\Program Files\Git\usr\bin\vendor_perl;C:\Program Files\Git\usr\bin\core_perl.

Boot class path: Unavailable

4. VisualVM

4.1. Снять график изменения показаний MBean-классов

открываем VisualVM и устанавливаем плагин VisualVM-MBeans. после чего в появившейся вкладке находим наши бины открываем графики и делаем пару кликов, потом повторяем процедуру со вторым бином.



4.2. определить имя класса, объекты которого занимают наибольший объём памяти JVM

заходим во вкладку Sampler

Overview Monitor Threads Sampler Profiler MBeans			
org.jboss.modules.Main (pid 13324)			
Sampler Settings			
Samp... CPU Memory Stop			
Status: memory sampling in progress			
Heap histogram Per thread allocations			
Results: Collected data: Snapshot		Perform GC Heap Dump	
Name	Live Bytes	Live Objects	
byte[]	39 160 024 B (24,1 %)	465 199	(12,7 %)
java.lang.Object[]	17 500 952 B (10,8 %)	451 105	(12,3 %)
java.util.TreeMap\$Entry	9 317 040 B (5,7 %)	232 926	(6,3 %)
java.util.HashMap\$Node	9 241 088 B (5,7 %)	288 784	(7,9 %)
java.lang.String	7 418 064 B (4,6 %)	309 086	(8,4 %)
int[]	5 852 752 B (3,6 %)	21 560	(0,6 %)
java.util.HashMap\$Node[]	4 985 224 B (3,1 %)	43 086	(1,2 %)
org.jboss.jandex.MethodInternal	3 333 216 B (2,1 %)	69 442	(1,9 %)
java.lang.Class	3 258 192 B (2,0 %)	27 326	(0,7 %)
char[]	3 115 976 B (1,9 %)	21 614	(0,6 %)
java.util.HashMap	2 597 040 B (1,6 %)	54 105	(1,5 %)
java.util.LinkedHashMap\$Entry	1 757 560 B (1,1 %)	43 939	(1,2 %)
java.lang.reflect.Method	1 599 488 B (1,0 %)	18 176	(0,5 %)
java.util.TreeMap\$KeyIterator	1 585 632 B (1,0 %)	49 551	(1,3 %)
java.util.jar.JarFile\$JarFileEntry	1 535 040 B (0,9 %)	14 760	(0,4 %)
java.util.ArrayList	1 498 800 B (0,9 %)	62 450	(1,7 %)

и видим что самый большой класс это byte.

4.3. определить пользовательский класс, в экземплярах которого находятся эти объекты

класс байт есть почти в каждом классе так что просто находим самый большой пользовательский класс.

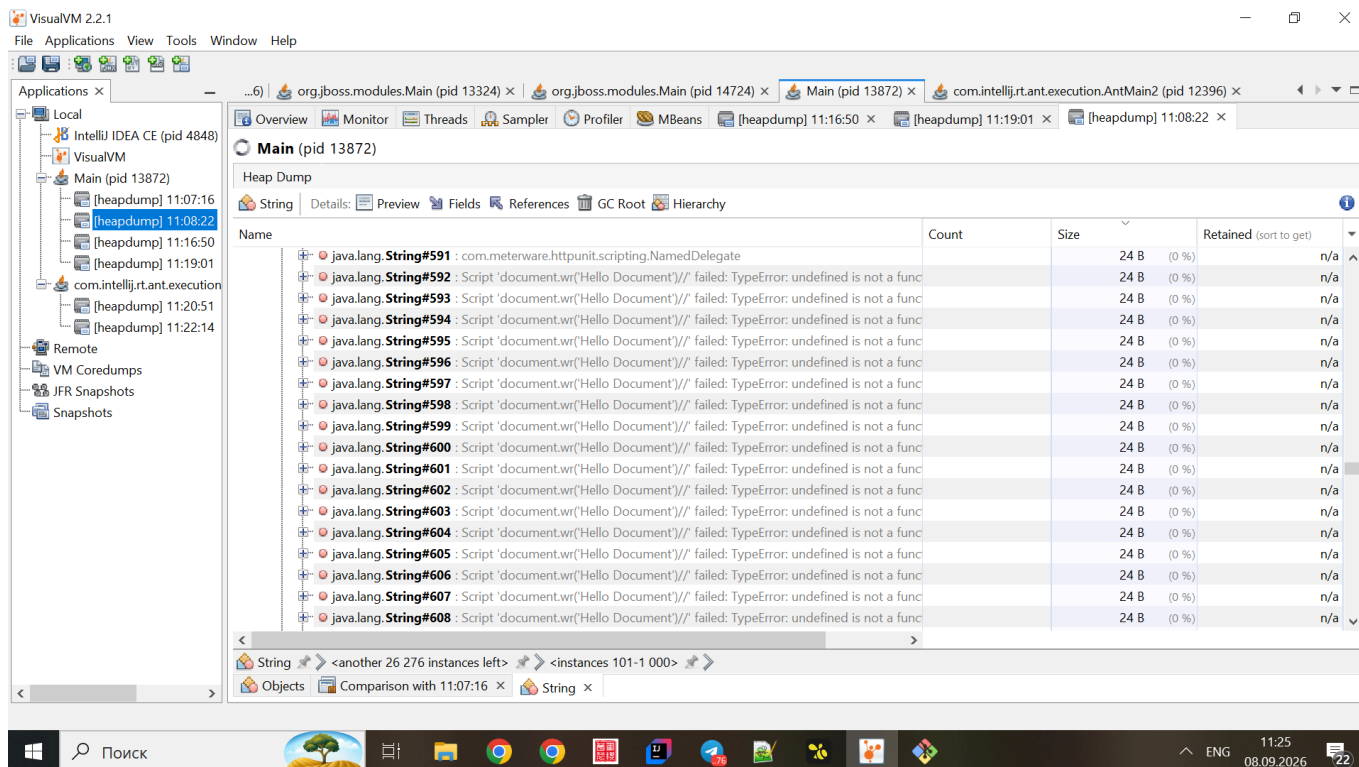
Start Page x org.jboss.modules.Main (pid 7056) x org.jboss.modules.Main (pid 13324) x			
Overview Monitor Threads Sampler Profiler MBeans [heapdump] 09:30:14 x			
org.jboss.modules.Main (pid 13324)			
Heap Dump			
Objects Preset: All Objects Aggregation: Details: Preview Fields References GC Root Hierarchy			
Name	Count	Size	Retained
org.hibernate.validator.internal.util.ConcurrentReferenceHashMap\$SoftKeyReference	2 (0 %)	80 B (0 %)	80 B (0 %)
org.hibernate.validator.internal.metadata.raw.ConstrainedElement\$ConstrainedElementKind[]	2 (0 %)	80 B (0 %)	80 B (0 %)
org.hibernate.validator.internal.metadata.core.AnnotationProcessingOptionsImp	2 (0 %)	80 B (0 %)	656 B (0 %)
org.hibernate.validator.internal.metadata.core.ConstraintHelper	2 (0 %)	80 B (0 %)	33 488 B (0 %)
org.hibernate.validator.internal.engine.constraintvalidation.ConstraintValidatorManagerImp	2 (0 %)	80 B (0 %)	240 B (0 %)
jakarta.faces.component.html.HtmlMessages\$PropertyKeys[]	1 (0 %)	80 B (0 %)	80 B (0 %)
com.sun.faces.facelets.tag.faces.core.ConvertDateTimeHandler	1 (0 %)	80 B (0 %)	96 B (0 %)
com.sun.faces.facelets.tag.faces.html.StylesheetResourceDelegate	2 (0 %)	80 B (0 %)	80 B (0 %)
com.sun.faces.util.MultiKeyConcurrentHashMap\$Segment[]	1 (0 %)	80 B (0 %)	12 888 B (0 %)
org.primefaces.shaded.owasp.encoder.CSSEncoder\$Mode	2 (0 %)	80 B (0 %)	80 B (0 %)
org.primefaces.shaded.owasp.encoder.URLEncoder\$Mode	2 (0 %)	80 B (0 %)	80 B (0 %)
org.primefaces.expression.SearchExpressionHint[]	2 (0 %)	80 B (0 %)	80 B (0 %)
app.backend.bean.InputBean	2 (0 %)	80 B (0 %)	560 B (0 %)
org.jboss.jca.adapters.jdbc.local.LocalManagedConnector	1 (0 %)	80 B (0 %)	1 048 B (0 %)
org.wildfly.elytron.web.undertow.server.ElytronContextAssociationHandle	2 (0 %)	80 B (0 %)	2 000 B (0 %)
org.wildfly.elytron.web.undertow.server.servlet.ElytronHttpServletExchange	2 (0 %)	80 B (0 %)	80 B (0 %)
org.h2.command.dml.TransactionCommand	1 (0 %)	80 B (0 %)	224 B (0 %)
org.h2.command.dml.Call	1 (0 %)	80 B (0 %)	344 B (0 %)
org.h2.command.dml.Delete	1 (0 %)	80 B (0 %)	704 B (0 %)

Object	Count	Size	Memory
app.backend.bean.InputBean	2 (0 %)	80 B (0 %)	560 B (0 %)
app.backend.bean.InputBean#1		40 B (0 %)	304 B (0 %)
<fields>			
static <resolved_references> = [] java.lang.Object[]#47030 : 35 items		160 B (0 %)	1 512 B (0 %)
[31] = java.lang.invoke.BoundMethodHandle\$Species_LLLLLLLLL#22		64 B (0 %)	408 B (0 %)
[19] = java.lang.invoke.MethodHandleImpl\$AsVarargsCollector#330		40 B (0 %)	176 B (0 %)
[30] = java.lang.invoke.BoundMethodHandle\$Species_LL#541		40 B (0 %)	112 B (0 %)
[32] = java.lang.invoke.BoundMethodHandle\$Species_LL#316		40 B (0 %)	104 B (0 %)
[0] = java.lang.String#142740 : Попадание		24 B (0 %)	64 B (0 %)
static <resolved_references> = [] java.lang.Object[]#853 : 32 items		144 B (0 %)	792 B (0 %)
value = [] byte[]#194920 : 18 items		40 B (0 %)	40 B (0 %)
[0] = byte 31		-	-
[1] = byte 4		-	-
[2] = byte 62		-	-
[3] = byte 4		-	-
[4] = byte 63		-	-
[5] = byte 4		-	-

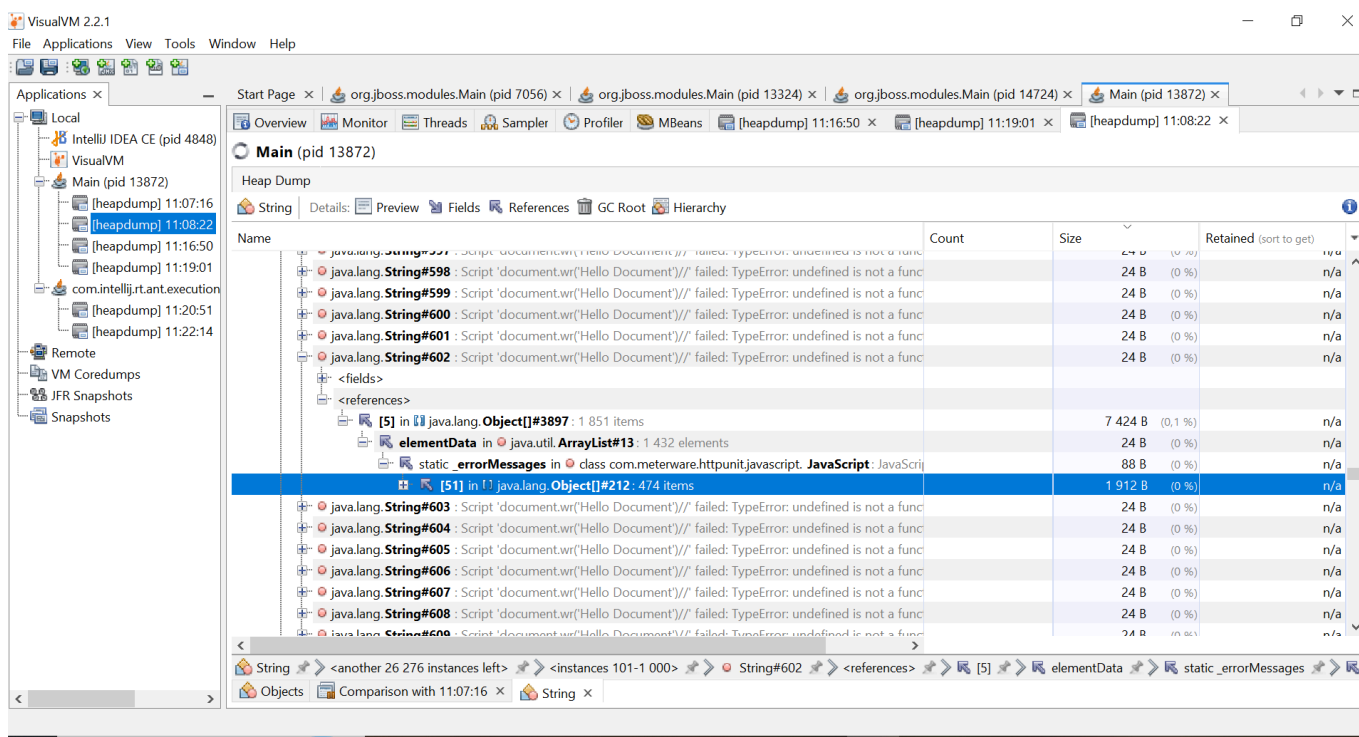
5. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе делаем два дампа с интервалом в минуту-две предварительно прожав Perform GC на вкладке монитор, затем сравниваем их

Name	Count	Size
java.util.TreeMap\$Entry	+856	+34 240 B
java.lang.Long	+416	+9 984 B
byte[]	+407	+42 248 B
java.lang.String	+407	+9 768 B
java.util.TreeMap	+272	+13 056 B
java.util.TreeMap\$KeySet	+272	+4 352 B
javax.management.openmbean.CompositeDataSupport	+272	+6 528 B
java.lang.Object[]	+128	+5 544 B
java.util.Arrays\$ArrayList	+128	+3 072 B
java.util.LinkedHashMap\$Entry	+128	+5 120 B
java.util.Collections\$UnmodifiableRandomAccessList	+128	+3 072 B
java.util.LinkedHashMap	+16	+1 024 B
java.util.HashMap\$Node[]	+16	+1 280 B
javax.management.openmbean.TabularDataSupport	+16	+384 B
java.lang.String[]	+16	+384 B
javax.management.ObjectName\$Property[]	+16	+384 B
javax.management.ObjectName\$Property	+16	+384 B

видим подозрительно большое кол-во строк и смотрим их



находим много однотипных сообщений об ошибке.

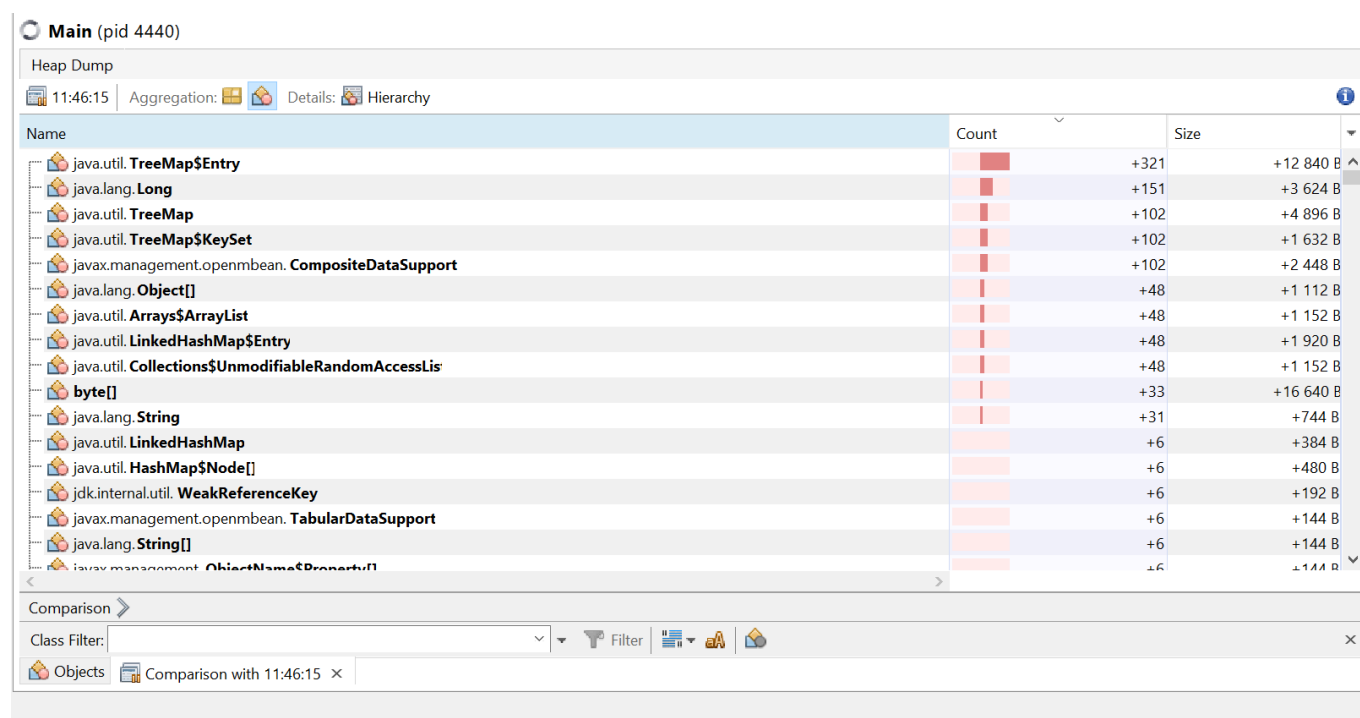


находим их источник. они создаются в классе com.meterware.httpunit.javascript и аккумулируются в динамическом массиве _errorMessages воспользовавшись функцией поиска находим метод clearErrorMessages который должен этот список очищать. идя вверх по иерархии с помощью функциональности IDEA доходим до метода clearScriptErrorMessages(в классе HttpUnitOptions)

который(метод) не используется, как нам подсказывает IDEA. добавляем вызов этого метода в рабочий цикл:

```
while (true) {  
    WebResponse response = sc.getResponse(request);  
    System.out.println("Count: " + number++ +  
response);  
    HttpUnitOptions.clearScriptErrorMessages();  
    java.lang.Thread.sleep(200);  
}
```

и проверяем программу повторно.



Name	Count	Size
java.util.TreeMap\$Entry	+321	+12 840 B
java.lang.Long	+151	+3 624 B
java.util.TreeMap	+102	+4 896 B
java.util.TreeMap\$KeySet	+102	+1 632 B
javax.management.openmbean.CompositeDataSupport	+102	+2 448 B
java.lang.Object[]	+48	+1 112 B
java.util.Arrays\$ArrayList	+48	+1 152 B
java.util.LinkedHashMap\$Entry	+48	+1 920 B
java.util.Collections\$UnmodifiableRandomAccessList	+48	+1 152 B
byte[]	+33	+16 640 B
java.lang.String	+31	+744 B
java.util.LinkedHashMap	+6	+384 B
java.util.HashMap\$Node[]	+6	+480 B
jdk.internal.util.WeakReferenceKey	+6	+192 B
javax.management.openmbean.TabularDataSupport	+6	+144 B
java.lang.String[]	+6	+144 B
javax.management.ObjectName\$Property[]	+6	+144 B

и замечаем значительное уменьшение объектов строки.

6. Заключение

при выполнении лабораторной работы я познакомился со средствами мониторинга и профилирования, а именно Mbeans, Jconsole, VisualVM и научился работать с ними для выявления проблем в производительности приложения.